

Engineering Notes

Multi-Tenant Realtime Ops Console
Take-Home Task · Node.js + Express + PostgreSQL + Socket.io + Cloudinary

Frontend	Next.js (App Router) + TypeScript
Backend	Node.js + Express
Database	PostgreSQL with Sequelize ORM
Realtime	Socket.io — rooms scoped per organisation
File Storage	Cloudinary (S3-compatible API via multer-cloudinary)
Auth	JWT — verified on every HTTP request and WebSocket connection
Search	PostgreSQL tsvector + GIN index + DB trigger for auto-update
Pagination	Cursor-based (base64 encoded timestamp)
Audit	Insert-only audit_logs table — no UPDATE or DELETE allowed
Rate Limits	3 tiers: auth (20/15 min), uploads (30/min), general API (200/15 min)

1. Multi-Tenancy Enforcement Strategy

Multi-tenancy means many different organisations share the same database and server, but each organisation's data is completely hidden from every other. This section explains exactly how that isolation is built into the code.

The core rule — every table has an org_id column

Every table that holds organisation-specific data — tickets, comments, attachments, audit_logs, org_members, invites — has an org_id column. This column is set when a record is created and never changed. Every single database query that reads or writes one of these tables includes org_id in the WHERE clause, like this:

```
WHERE id = :ticketId AND org_id = :orgId
```

This means even if a user knows the exact UUID of a ticket that belongs to another organisation, the query returns nothing because the org_id from their token will not match.

The two middleware layers

Every API request that touches organisation data goes through two checks before it reaches the actual controller:

Reads the Bearer JWT token from the Authorization header. Verifies the signature and expiry. Then loads the user fresh from the database to catch deactivated accounts. If anything fails the request is rejected with 401 immediately — no data is ever touched.

Reads orgId from the URL parameter. Checks that the authenticated user has a row in org_members for that org. Then checks their role level meets the minimum required for that specific route. A Viewer trying to create a ticket gets 403 here, before any DB write is attempted.

Role hierarchy

Roles are compared as numbers so the check is one simple comparison:

viewer	0	Read-only. Can see tickets and comments but cannot change anything.
member	1	Can create and edit tickets, post comments, and upload files.
admin	2	Can manage org members and change roles in addition to member actions.
owner	3	Full control. Cannot be removed from the org by anyone else.

Organisation switching

When a user switches organisations on the frontend, the auth store updates and all subsequent API calls use the new orgId from the URL. On the WebSocket side, the client leaves the previous org room and joins the new one. The server re-checks org membership before allowing the join, so a user cannot stay in an org room after being removed.

✓ Security summary: org_id always comes from the verified JWT, never from the request body. A user cannot lie about which org they belong to.

2. Realtime Security — No Cross-Org Leaks

The realtime system uses Socket.io. The main risk with WebSockets is that an event meant for Organisation A accidentally reaches a user in Organisation B. Here is how that is prevented at every step.

Connection authentication

When the frontend opens a WebSocket connection, it sends its JWT inside the handshake auth object. The Socket.io middleware in `socketHandler.js` runs before the connection is accepted:

```
const token = socket.handshake.auth?.token;
```

The token is verified and the user is loaded from the database. If the token is missing, expired, or the user is inactive, the connection is rejected before any events can flow. The user object is then attached to the socket and available for all future events on that connection.

Org rooms — the key isolation mechanism

After connecting, the frontend emits a `join_org` event with the `orgId`. The server does NOT just trust this value. It queries the `org_members` table to confirm the user is actually a member of that org. Only then is the socket added to a room named `org:{orgId}`. All events — ticket changes, new comments, attachment uploads, presence updates — are emitted only to that specific room:

```
req.io?.to(`org:${orgId}`).emit("ticket:created", { ticket })
```

A user in org-A's room will never receive an event emitted to org-B's room. Socket.io handles the delivery isolation, and the membership check handles the access control.

Typing indicators and presence

Typing indicators (`user_typing`, `user_stopped_typing`) and the presence list (who is currently online) are also scoped to the org room. The server reads `socket.currentOrgId` — which was set at join time after the membership check — so a user cannot fake which org room they broadcast to.

Organisation switching on the socket

When a user switches orgs, the socket handler leaves all previous org rooms and removes the user from the presence map of those orgs before joining the new room. This means they stop receiving events from the old org immediately.

■ **Known limitation:** Presence data is stored in memory on the Node.js process. If the app is scaled to multiple server instances, presence will not be shared across them. Adding a Redis adapter for Socket.io would fix this.

3. Search and Indexing Decisions

The app needs to search through 10,000+ tickets quickly. This section explains the decisions made to keep search fast.

Full-text search with PostgreSQL tsvector

Rather than using LIKE '%keyword%' queries — which are very slow on large tables because they cannot use indexes — the tickets table has a search_vector column of type TSVECTOR. This is a pre-processed, indexed version of the text content. A database trigger keeps it automatically up to date whenever a ticket is saved:

```
NEW.search_vector := setweight(to_tsvector('english', coalesce(NEW.title, '')), 'A') ||  
setweight(to_tsvector('english', coalesce(NEW.description, '')), 'B');
```

The title gets weight A (most important) and description gets weight B. This means a search for 'database' will rank a ticket with that word in the title higher than one that only mentions it in the description.

Searching then uses the @@ operator which is extremely fast because it uses the index:

```
search_vector @@ plainto_tsquery('english', 'database timeout')
```

GIN index for full-text search

A GIN (Generalised Inverted Index) index is placed on the search_vector column. GIN is the standard choice for tsvector columns in PostgreSQL. It maps each word to the list of tickets containing it, so a word lookup is instant regardless of how many tickets exist.

Filter indexes

For filtered queries — by status, severity, assignee, tags, or date range — composite indexes are added with org_id as the leading column, because every single query is scoped to one org:

(org_id)	Base filter — every query starts here
(org_id, status)	Filter tickets by status within an org
(org_id, severity)	Filter tickets by severity within an org
(assignee_id)	Find all tickets assigned to a user
(created_at)	Sorting and cursor pagination
(org_id, created_at)	Audit log pagination per org
(entity_type, entity_id)	Audit log — full history of one ticket
GIN on tags array	Fast contains check: find tickets with all specified tags
GIN on search_vector	Full-text search

Tag search

Tags are stored as a PostgreSQL text array column. A GIN index on this column allows the @> (contains) operator to run fast even with many unique tags. The filter finds tickets that contain ALL of the requested tags, not just any of them:

```
WHERE tags @> ARRAY['infra', 'database']
```

4. Cursor Pagination Design

There are two ways to paginate a large list of results. This section explains why cursor pagination was chosen over the simpler offset approach.

Why not LIMIT / OFFSET?

The simple approach is LIMIT 25 OFFSET 100, which means 'skip the first 100 rows, give me the next 25'. The problem is that PostgreSQL still has to read and count those 100 skipped rows every time. At page 200 of a 10,000-ticket list, it must scan 5,000 rows just to throw them away. This gets slower the deeper you go.

How cursor pagination works here

Instead of a page number, the client receives a cursor — a pointer to the last item it saw. The next request sends this cursor back, and the query fetches only records that come after that point:

```
WHERE org_id = :orgId AND created_at < :lastSeenTimestamp ORDER BY created_at DESC LIMIT 25
```

Because created_at is indexed, this query goes straight to the right position in the index without scanning anything that came before it. It is just as fast on page 400 as it is on page 1.

The cursor format

The cursor is the ISO timestamp of the last ticket in the current page, base64 encoded to make it opaque to the client:

```
// Create cursor const nextCursor =  
Buffer.from(lastTicket.created_at.toISOString()).toString('base64'); // Decode cursor on  
next request const timestamp = new Date(Buffer.from(cursor, 'base64').toString('utf8'));
```

The base64 encoding is just to avoid the client treating it as a meaningful timestamp they could manipulate. It is not encryption — it just makes it look like a token.

The +1 trick for hasMore

To know whether there is a next page without running a second COUNT query, the code fetches limit + 1 rows. If it gets back 26 when the limit is 25, there are more results — it returns only 25 and sets hasMore to true. If it gets back 25 or fewer, there is no next page.

■ Trade-off: Cursor pagination does not support jumping to a specific page number. The user can only go forward (load more). For an ops console where users scroll through recent incidents, this is acceptable behaviour.

5. File Storage — Cloudinary

The task specified S3-compatible file storage. This project uses Cloudinary, which provides an S3-compatible API. The backend code uses the AWS SDK and `multer-s3` pointed at Cloudinary's endpoint — no Cloudinary-specific SDK is needed.

How uploads work

Files are uploaded via the backend — the browser sends the file to the Express server, and `multer` streams it directly to Cloudinary without writing anything to disk. This is done using `multer-s3` with the Cloudinary S3-compatible endpoint as the target.

`attachments/{orgId}/{ticketId}/{uuid}.{ext}` — files are organised by org and ticket, making it easy to find or bulk-delete all files for a given ticket.

Only specific MIME types are accepted: JPEG, PNG, GIF, WebP, PDF, plain text, ZIP, and Word documents. Any other type is rejected before upload begins.

Maximum 20 MB per file. Maximum 5 files per upload request. These limits are enforced by `multer` before any data is streamed to Cloudinary.

Access control — signed URLs

Files stored on Cloudinary are private. To download a file, the client requests a signed URL from the backend. The backend first checks that the attachment belongs to the user's org, then generates a pre-signed URL valid for 15 minutes:

```
const url = await getSignedUrl(s3Client, command, { expiresIn: 900 });
```

The client never has a permanent direct link to the file. Every download requires going through the backend auth check, which prevents users from sharing links to files from other organisations.

Delete flow

When a file is deleted, two things happen: the file is removed from Cloudinary using the `DeleteObjectCommand`, and the attachment record is deleted from the database. Only the original uploader or an admin/owner can delete a file — this check is done in the controller before either deletion step runs.

■ Trade-off: Files go through the Node.js backend before reaching Cloudinary. Under high concurrency with large files, this ties up the event loop. The production fix would be presigned upload URLs — the browser uploads directly to Cloudinary, bypassing the server entirely.

6. Audit Logging — Immutable Trail

Every important action in the system — creating a ticket, changing a status, uploading a file, inviting a member — is recorded in the `audit_logs` table. This log can never be modified or deleted.

What is recorded

<code>ticket.created</code>	New ticket with full initial data in <code>new_data</code>
<code>ticket.updated</code>	Old field values in <code>old_data</code> , new values in <code>new_data</code>
<code>ticket.status_changed</code>	Previous status in <code>old_data</code> , new status in <code>new_data</code>
<code>ticket.deleted</code>	Full ticket snapshot in <code>old_data</code> , <code>new_data</code> is null
<code>attachment.uploaded</code>	File count and <code>ticket_id</code> in <code>new_data</code>
<code>attachment.deleted</code>	File name in <code>old_data</code> , <code>new_data</code> is null
<code>member.invited</code>	Invite details including target email and role
<code>member.role_changed</code>	Old role in <code>old_data</code> , new role in <code>new_data</code>

How immutability is enforced

The `AuditLog` Sequelize model has `beforeUpdate` and `beforeDestroy` hooks that throw an error if anything tries to change or delete a log entry:

```
beforeUpdate: () => { throw new Error('Audit logs are immutable'); }, beforeDestroy: () => { throw new Error('Audit logs are immutable'); },
```

The table also has no `updatedAt` column — it only has `created_at` — which makes it visually clear that these records never change. The `updatedAt: false` option in the model config prevents Sequelize from even trying to set that column.

Audit service design

All audit writes go through a single `auditService.log()` function. This function wraps the database insert in a `try/catch`. If the audit write fails for any reason, it logs the error but does NOT throw — so an audit failure never crashes the main request or rolls back the actual action. The user's operation succeeds regardless.

Each log entry contains

- org_id** — Which organisation this action belongs to
- actor_id** — Which user performed the action (null for system events)
- action** — Dot-namespaced action name, e.g. `ticket.status_changed`
- entity_type + entity_id** — What record was affected
- old_data / new_data** — Full JSONB snapshots of before and after state

ip_address + user_agent — Request metadata for security investigations

7. Scaling Bottlenecks

This section honestly describes the parts of the system that would struggle under heavy load and what the fix would be.

WebSocket presence — in-memory state

The presence system (who is currently online) stores data in a JavaScript Map in memory on the Node.js process. This works fine for one server instance. However if the app is scaled horizontally to multiple instances behind a load balancer, a user connected to Server A would not appear in the presence list on Server B, and events emitted on Server A would not reach users on Server B.

■ Fix: Add the Socket.io Redis adapter. All instances share one Redis pub/sub channel, so events and presence data are automatically broadcast across all servers.

File uploads through the backend

Currently files travel: browser → Express server → Cloudinary. For large files under high concurrency, this ties up the Node.js event loop while bytes are streaming through.

■ Fix: Use presigned upload URLs. The backend generates a signed URL, the browser uploads directly to Cloudinary, and only the metadata (filename, size, key) is sent back to the backend. The server never touches the file bytes.

Full-text search ceiling

PostgreSQL tsvector search works very well up to several million rows on a properly indexed single database. Beyond that, or if features like fuzzy matching, typo tolerance, or relevance ranking are needed, a dedicated search service like Elasticsearch or Typesense would be more appropriate.

Database connection pool

The backend uses a PostgreSQL connection pool via Sequelize. Under very high concurrent request load the pool can be exhausted, causing requests to queue waiting for a free connection. Adding PgBouncer as a connection pooler in front of PostgreSQL would allow many more application connections to share fewer actual database connections efficiently.

Audit log table growth

The audit_logs table is insert-only and grows forever. With many active users and frequent ticket changes, this table can become very large. PostgreSQL table partitioning by created_at (e.g. monthly partitions) would keep query performance stable as the table grows to tens of millions of rows.

8. Trade-offs and Missing Features

This section honestly describes what was not fully built, what shortcuts were taken, and what the next steps would be.

No Redis

There is no Redis in this setup. This means no Socket.io multi-instance support, no shared rate limit counters across processes, and no caching layer for expensive queries. Redis would be the first infrastructure addition before any real production deployment.

Email invites not sent

The invite system creates a token and stores it in the database, but no actual email is sent. A production version would integrate an email service such as Resend or SendGrid. For now, the invite link would need to be shared manually.

Rate limiting is per-IP only

express-rate-limit tracks requests by IP address. This means a single determined user behind a corporate NAT (sharing one IP with many colleagues) could hit limits unfairly. A better approach is per-user rate limiting using the `user_id` from the JWT, combined with per-IP limits as a fallback.

No automated tests

There is no test suite. Unit tests for controllers (especially the multi-tenancy `org_id` checks) and integration tests for the auth middleware chain would be the first things added before calling this production-ready.

Search does not cover comments

The tsvector search index is built from ticket title and description only. Comment content is not indexed. Adding comment search would require either including comment text in the ticket's search vector (updated on every new comment) or a separate search index on the comments table.

Optimistic UI is partial

Comments appear in the UI immediately before the server confirms (optimistic). But ticket status changes and edits wait for the server response before updating. Full optimistic updates with rollback on failure would make the UI feel faster.

Audit log has no UI yet

The `audit_logs` table records every action correctly. However there is no page in the frontend to browse or search the audit history. Building an admin-only audit log view was descope'd for time.

No end-to-end encryption

Data is encrypted in transit (HTTPS/WSS) and at rest by the database and Cloudinary. But there is no application-level field encryption for sensitive ticket content. For highly sensitive industries this might be a requirement.

9. Quick Reference

Topic	Decision	Reason
Multi-tenancy	org_id on every table + middleware checks	Simple, reliable, enforced at DB level
Auth	JWT Bearer token	Stateless — scales without session storage
WebSocket auth	JWT in handshake, verified before connect	Never authenticated sockets ever connect
Org isolation (WS)	Rooms named org:{orgId}	Socket.io handles delivery isolation
Full-text search	tsvector + GIN index + DB trigger	Fast, native to PostgreSQL, no extra service
Tag search	ARRAY column + GIN index + @> operator	Efficient multi-tag containment check
Pagination	Cursor-based (base64 timestamp)	Consistent performance at any depth
File storage	Cloudinary via S3-compatible API	Simple integration, no S3 account needed
File access	Signed URLs, 15 min expiry	Files stay private, access requires auth
Audit logs	Insert-only, JSONB old/new data	Full history, tamper-proof, searchable
Rate limiting	3 tiers by route type	Stricter on auth routes, looser on reads
Logging	Winston + Morgan	Structured logs, easy to ship to log aggregator